

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC MÁY Phase 2

Sinh viên thực hiện:

Phan Bá Đức - 22120071

Nguyễn Bá Duy - 22120078

Đặng Duy Lâm - 22120182

Giảng viên hướng dẫn:

TS. Bùi Duy Đăng

ThS. Nguyễn Thanh Tình

TP. Hồ Chí Minh, tháng 6 năm 2025.

Mục lục

A.	Thông tin các thành viên và tự đánh giá.....	2
B.	Tóm tắt bài toán.	2
C.	Ý tưởng giải quyết.....	3
1.	Tiền xử lý.....	3
2.	Xây dựng mô hình và đánh giá.....	3
3.	Huấn luyện mô hình.....	4
a.	Khai báo mô hình	4
b.	Khai báo các tham số cho Grid Search	4
c.	Train mô hình và siêu tham số tốt nhất	5
4.	Đánh giá mô hình.....	5
a.	Các chỉ số hiệu suất.....	6
b.	Biểu đồ tương quan giữa Accuracy, Tổng kích thước nơ-ron trong các tầng ẩn và Bộ nhớ sử dụng.....	7
c.	Ma trận nhầm lẫn.....	7
d.	Phân tích hạn chế.....	8
e.	Thảo luận một số cách tiếp cận khác.....	8
D.	Kết quả trên tập test.....	10
	Tham Khảo	10

A. Thông tin các thành viên và tự đánh giá.

- Thông tin các thành viên:

STT	Họ và tên	Mã số sinh viên	Mức độ hoàn thành
1	Phan Bá Đức	22120071	100%
2	Nguyễn Bá Duy	22120078	100%
3	Đặng Duy Lâm	221200182	100%

- Tự đánh giá mức độ hoàn thành:

STT	Vấn Đề	Đánh giá
1	Tìm hiểu về dữ liệu, đặt vấn đề và tìm giải pháp tương ứng.	60%
2	Phát triển và huấn luyện mô hình	20%
3	Đánh giá mô hình và lựa chọn ra mô hình phù hợp nhất.	20%
4	Tổng quát	100%

B. Tóm tắt bài toán.

Bài toán đặt ra trong cuộc thi yêu cầu xây dựng một mô hình học máy nhằm nhận diện âm thanh đặc trưng của loài gà tây dựa trên tập các đặc trưng âm thanh đã được trích xuất sẵn. Dựa vào nhãn của tập dữ liệu is_turkey, đây là một bài toán phân loại nhị phân, trong đó mỗi mẫu âm thanh cần được dự đoán là có hoặc không thuộc về gà tây.

Dữ liệu được cung cấp bao gồm các đặc trưng sau:

vid_id	Video ID liên quan đến mẫu dữ liệu này
audio_embedding	Đặc trưng âm thanh theo từng frame, giảm chiều xuống còn 128 chiều bằng VGGish
start_time_seconds_youtube_clip	Thời điểm bắt đầu của đoạn audio trong video YouTube (tính bằng giây)
end_time_seconds_youtube_clip	Thời điểm kết thúc của đoạn audio trong video YouTube (tính bằng giây)
is_turkey	Nhãn mục tiêu: 1 nếu đoạn audio chứa tiếng gà tây, 0 nếu không

C. Ý tưởng giải quyết.

1. Tiền xử lý.

Trong bài toán này, hai thuộc tính chính được giữ lại để huấn luyện mô hình là `audio_embedding` (đặc trưng âm thanh) và `is_turkey` (nhãn phân loại). Các thuộc tính khác được loại bỏ vì không đóng góp nhiều thông tin cho quá trình học.

Tuy nhiên, một vấn đề phát sinh từ thuộc tính `audio_embedding` là nó có dạng ba chiều với kích thước (số mẫu, số frame, 128), trong khi nhãn `is_turkey` chỉ có kích thước (số mẫu, 1). Ngoài ra, số lượng frame không giống nhau giữa các mẫu, dẫn đến việc không thể xử lý dữ liệu theo cách thông thường. Do đó, nhóm đưa ra hướng giải quyết như sau:

- Mỗi mẫu với kích thước (frame, 128) được làm phẳng thành một vector một chiều có kích thước (frame $\times$ 128).
- Do số frame ở mỗi mẫu là khác nhau, cần thêm padding 0 để tất cả các mẫu có cùng số chiều, đảm bảo dữ liệu có thể đưa vào mô hình.

```
class AudioDataset(Dataset):
    def __init__(self, data, T_max=None):
        """
        Hàm khởi tạo cho lớp AudioDataset
        :param
            data: Dữ liệu đầu vào, mỗi phần tử là một dictionary chứa thông tin về video
            T_max: Số lượng frame tối đa cho mỗi mẫu. Nếu không cung cấp, sẽ lấy giá trị lớn nhất từ dữ liệu.
        """
        self.data = data
        self.T_max = T_max if T_max is not None else max(len(sample['audio_embedding']) for sample in data)

        # Tiền xử lý dữ liệu: padding các embedding âm thanh để đảm bảo chúng có cùng kích thước
        for sample in data:
            embedding = sample['audio_embedding']
            len_emb = len(embedding)
            # Nếu số lượng frame của một mẫu nhỏ hơn T_max, sẽ thêm padding bằng 0
            if len_emb < self.T_max:
                padding = [[0] * 128] * (self.T_max - len_emb)
                embedding.extend(padding)

            # Chuẩn hóa dữ liệu: chuyển đổi embedding thành tensor và chia cho 255.0 để đưa về khoảng [0, 1]
            feature = torch.tensor(embedding, dtype=torch.float32) / 255.0

            # Lưu trữ feature đã được flatten vào sample
            sample['feature'] = feature.flatten()
```

2. Xây dựng mô hình và đánh giá.

Tập dữ liệu trong file `train.json` gồm 1195 mẫu. Nhóm em chia ra thành hai tập là tập `train` (80%) và tập `validation` (20%)

Nhãn của dữ liệu có hai lớp: 704 mẫu thuộc lớp 0 và 491 mẫu thuộc lớp 1. Có một sự chênh lệch ở đây

Cuối cùng, nhóm quyết định sẽ kết hợp Grid để tìm ra các siêu tham số tối ưu, gồm số lượng tầng ẩn và số nơ-ron mỗi tầng. Ngoài ra có thể tìm learning rate nhưng nhóm hiện tại đang dùng Adam (có khả năng tự điều chỉnh learning rate) làm optimizer nên sẽ để số learning rate trong Grid là 1. Để sau này nếu có chuyển qua optimizer khác thì có thể thêm các learning rate khác vào Grid. Tiêu chí chính dùng để đánh giá là độ chính xác (accuracy), ngoài ra có F1-score, Precision, Recall, AUC-ROC, thời gian huấn luyện, bộ nhớ sử dụng

3. Huấn luyện mô hình.

a. Khai báo mô hình

```
class SimpleMLP(nn.Module):
    def __init__(self, input_size, hidden_sizes):
        """
        Hàm khởi tạo cho mô hình MLP
        :param
        input_size: Kích thước đầu vào của mô hình (số lượng đặc trưng đầu vào)
        hidden_sizes: Danh sách các kích thước của các lớp ẩn
        """
        super(SimpleMLP, self).__init__()
        layers = []
        in_size = input_size
        # Tạo các lớp ẩn dựa trên kích thước đã cho
        # Mỗi lớp bao gồm một lớp Linear, BatchNorm1d, ReLU và Dropout
        for out_size in hidden_sizes:
            layers.append(nn.Linear(in_size, out_size))
            layers.append(nn.BatchNorm1d(out_size))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(0.3))
            in_size = out_size
        # Lớp đầu ra cuối cùng, chỉ có một đầu ra
        layers.append(nn.Linear(in_size, 1))
        # Xếp các lớp theo kiểu tuần tự
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

Đầu vào:

- **Input_size**: kích thước đầu vào của mô hình (số lượng đặc trưng đầu vào), trong bài toán này là 1280
- **Hidden_sizes**: danh sách các kích thước các lớp ẩn

Các khai báo này nhằm mục đích tạo mô hình MLP với khả năng thay đổi số lượng lớp ẩn cũng như số lượng nơ-ron, phù hợp với nhiều bài toán và tìm kiếm siêu tham số tối ưu qua Grid Search

Khởi tạo list layers: tạo một danh sách rỗng để lưu các lớp mạng trước khi xếp thành chuỗi.

nn.Linear(in_size, out_size): Thêm một lớp tuyến tính (fully connected layer) với đầu vào in_size và đầu ra out_size.

nn.BatchNorm1d(out_size): Thêm lớp chuẩn hóa batch (Batch Normalization) cho đầu ra của lớp tuyến tính, giúp ổn định huấn luyện bằng cách chuẩn hóa dữ liệu đầu vào của từng mini-batch => Giảm biến thiên nội bộ, tăng tốc độ hội tụ và cải thiện khả năng tổng quát hóa.

nn.ReLU(): Thêm hàm kích hoạt ReLU

nn.Dropout(0.3): Thêm lớp Dropout với tỷ lệ 0.3, ngẫu nhiên tắt 30% nơ-ron trong quá trình huấn luyện để ngăn overfitting.

nn.Sequential(*layers): Tạo một mô-đun tuần tự từ danh sách layers, cho phép truyền dữ liệu qua các lớp theo thứ tự.

b. Khai báo các tham số cho Grid Search

```
# Định nghĩa không gian tham số cho Grid Search
num_layers_options = [2, 3, 4] # Số tầng ẩn
hidden_size_options = [64, 128, 256, 512] # Kích thước tầng ẩn
learning_rates = [1e-3] # Sử dụng 1 learning rate vì ta dùng optimizer là Adam có khả năng tự điều chỉnh LR
num_epochs = 100 # Số epoch tối đa
patience = 10 # Early stopping: dừng nếu không cải thiện sau 10 epoch
```

Tổng số phép thử là (công thức như trong ảnh)

```
# Tính tổng số vòng lặp thử nghiệm
total_trials = sum(len(hidden_size_options)**num_layers for num_layers in num_layers_options) * len(learning_rates)

[ ] print('Tổng số vòng lặp thử nghiệm:', total_trials)

TỔNG SỐ VÒNG LẶP THỬ NGHIỆM: 336
```

c. Train mô hình và siêu tham số tốt nhất

Nhóm em có dùng Early stopping để dừng mô hình nếu tham số không cải thiện sau n epoch. Điều này giúp tiết kiệm thời gian

Train mô hình với từng tổ hợp siêu tham số trong Grid Search và đánh giá trên tập validation. Sau đó lưu lại mô hình có kết quả accuracy tốt nhất trên tập val.

Qua từng nhóm tổ hợp siêu tham số mà mô hình thử qua, nhóm em đều lưu lại thời gian huấn luyện trên tổ hợp đó, bộ nhớ sử dụng và accuracy

Dưới đây là kết quả xuất ra ở những dòng cuối

```
-----
Early stopping tại epoch 11 ở cấu hình: [512, 512, 512, 128], LR: 0.001
Trial 334/336 - Cấu hình: [512, 512, 512, 128], LR: 0.001, Accuracy: 0.9456
Peak Memory: 59.52 MB
-----
Early stopping tại epoch 13 ở cấu hình: [512, 512, 512, 256], LR: 0.001
Trial 335/336 - Cấu hình: [512, 512, 512, 256], LR: 0.001, Accuracy: 0.9414
Peak Memory: 61.28 MB
-----
Early stopping tại epoch 33 ở cấu hình: [512, 512, 512, 512], LR: 0.001
Trial 336/336 - Cấu hình: [512, 512, 512, 512], LR: 0.001, Accuracy: 0.9540
Peak Memory: 64.81 MB
-----
Cấu hình tốt nhất: {'num_layers': 4, 'hidden_sizes': [64, 64, 64, 512], 'lr': 0.001}, Accuracy tốt nhất: 0.9665
```

4. Đánh giá mô hình

```
[16] print(f"Tổng thời gian huấn luyện: {end_time - start_time:.2f} giây")
      print(f"Thời gian trung bình {(end_time - start_time) / total_trials:.2f} giây")

TỔNG THỜI GIAN HUẤN LUYỆN: 554.93 giây
THỜI GIAN TRUNG BÌNH 1.65 giây

[17] print(f"Bộ nhớ trung bình được sử dụng: {np.mean(peak_memories):.2f} MB")

BỘ NHỚ TRUNG BÌNH ĐƯỢC SỬ DỤNG: 52.74 MB

[18] indice_best_acc = np.argmax(best_acc_list)
      print(f"Cấu hình tốt nhất sử dụng bộ nhớ: {peak_memories[indice_best_acc]:.2f} MB")

CẤU HÌNH TỐT NHẤT SỬ DỤNG BỘ NHỚ: 55.60 MB
```

Sau khi hoàn tất bước huấn luyện và lựa chọn siêu tham số bằng Grid Search, mô hình MLP có siêu tham số tối ưu là gồm 4 lớp ẩn với số lượng nơ-ron lần lượt là [64, 64, 512, 512] và

learning rate là 0.001. Độ chính xác cao nhất đạt được là 0.9665 (tức 96.65%) và bộ nhớ tốn 55.60 MB

Bây giờ, nhóm sẽ xem chi tiết performance của mô hình trên tập validation bằng cách chạy lại mô hình trên tập validation để đưa ra đánh giá cuối cùng

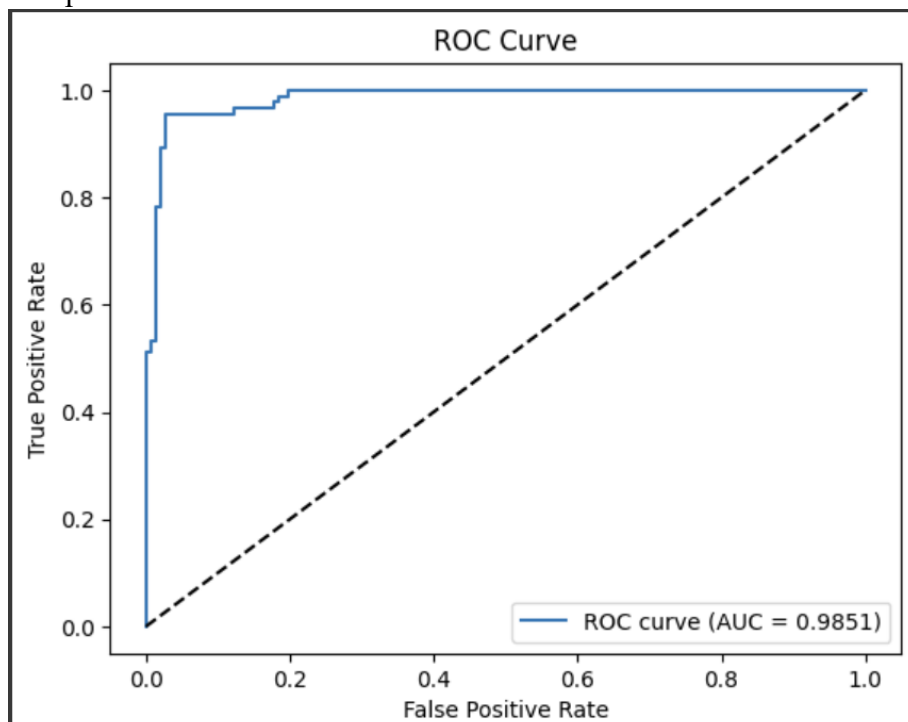
a. Các chỉ số hiệu suất

```
[19] accuracy = accuracy_score(val_labels, val_labels_preds)
precision = precision_score(val_labels, val_labels_preds)
recall = recall_score(val_labels, val_labels_preds)
auc_roc = roc_auc_score(val_labels, val_prob_preds) # Sử dụng xác suất để tính AUC-ROC

# In kết quả
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"AUC-ROC: {auc_roc:.4f}")
```

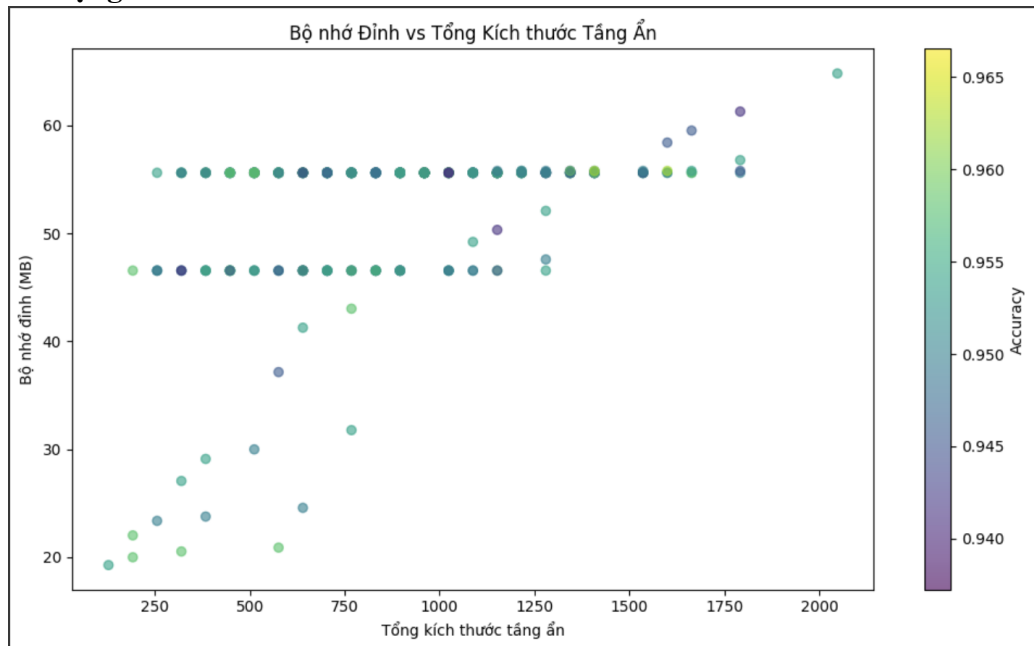
Accuracy: 0.9665
Precision: 0.9565
Recall: 0.9565
AUC-ROC: 0.9851

- **Accuracy:0.9665**
Mô hình đạt độ chính xác 96.65%, cho thấy khả năng phân loại tổng thể rất tốt.
- **Precision:0.9565**
Trong số các mẫu được dự đoán là lớp dương (1), 95.65% thực sự là dương, cho thấy mô hình ít dự đoán sai dương tính (false positive).
- **Recall:0.9565**
Mô hình phát hiện được 97.03% các mẫu dương thực sự, thể hiện khả năng nhận diện lớp dương rất cao.
- **AUC-ROC:0.9851**
Diện tích dưới đường cong ROC là 0.9851, gần với 1, cho thấy mô hình có khả năng phân biệt giữa hai lớp rất tốt.



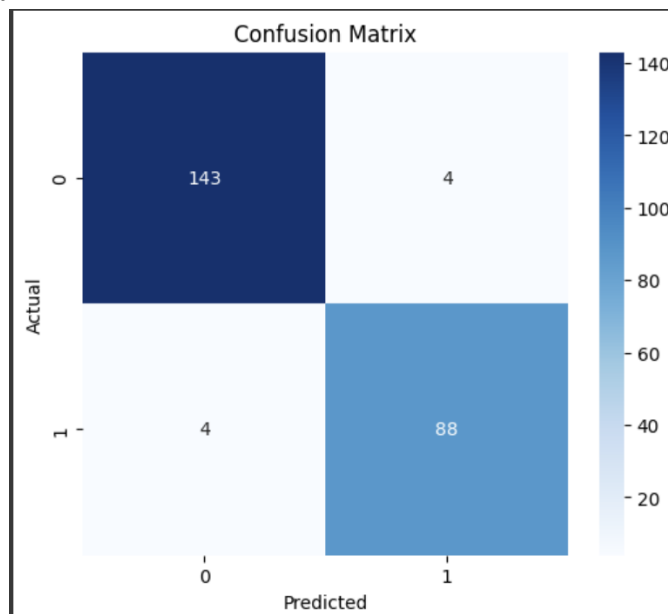
- Đường cong ROC gần sát góc trên bên trái, với $AUC = 0.9851$, xác nhận mô hình có khả năng phân biệt lớp rất tốt.
- Đường cong nằm trên đường chéo (random guess), cho thấy hiệu suất vượt trội so với dự đoán ngẫu nhiên.

b. Biểu đồ tương quan giữa Accuracy, Tổng kích thước nơ-ron trong các tầng ẩn và Bộ nhớ sử dụng



- Bộ nhớ đỉnh dao động từ ~20 MB đến ~65 MB, tăng dần khi tổng kích thước tầng ẩn tăng từ 250 đến 2000.
- Accuracy dao động từ 0.940 đến 0.965, với các cấu hình có kích thước tầng ẩn lớn hơn thường đạt accuracy cao hơn (đặc biệt ở mức 0.965).
- Xu hướng: Các cấu hình sử dụng nhiều bộ nhớ hơn (kích thước tầng ẩn lớn) có xu hướng đạt hiệu suất tốt hơn, nhưng cũng có ngoại lệ (một số cấu hình với kích thước nhỏ vẫn đạt accuracy cao ~0.960).

c. Ma trận nhầm lẫn



- True Negatives (0,0): 143 mẫu thuộc lớp 0 được dự đoán đúng.
- False Positives (0,1): 4 mẫu lớp 0 bị dự đoán sai thành lớp 1.
- False Negatives (1,0): 4 mẫu lớp 1 bị dự đoán sai thành lớp 0.
- True Positives (1,1): 88 mẫu thuộc lớp 1 được dự đoán đúng.

d. Phân tích hạn chế

1. Xác định lỗi

False Positives (5 mẫu): 4 mẫu lớp 0 bị dự đoán thành lớp 1. Điều này có thể do dữ liệu của lớp 0 có đặc trưng gần giống lớp 1, hoặc mô hình quá nhạy với các đặc trưng của lớp 1.

False Negatives (3 mẫu): 4 mẫu lớp 1 bị dự đoán thành lớp 0. Có thể do dữ liệu lớp 1 không đủ đa dạng, hoặc mô hình thiếu khả năng phát hiện các đặc trưng đặc trưng của lớp 1 trong các mẫu này.

2. Lý do (dự đoán)

Dữ liệu không cân bằng: Tổng số mẫu lớp 0 (147) và lớp 1 (92) cho thấy dữ liệu hơi lệch về lớp 0, có thể ảnh hưởng đến khả năng dự đoán lớp 1. Hoặc một phần cũng có thể do dữ liệu tập ban đầu đã không cân bằng giữa mẫu 0 và 1

Đặc trưng chưa tốt: Các đặc trưng hiện tại có thể không đủ để phân biệt tốt giữa các mẫu gần ranh giới quyết định.

Mô hình chưa tối ưu: Cấu hình MLP có thể chưa đủ phức tạp để xử lý các mẫu khó (cần thêm tầng, tăng kích thước tầng ẩn, hoặc thử kiến trúc khác như CNN/RNN).

e. Thảo luận một số cách tiếp cận khác

1. Kỹ thuật skip connection

```
# Định nghĩa mô hình MLP với Skip Connections
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size1=512, hidden_size2=128, hidden_size3=128):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size1)
        self.bn1 = nn.BatchNorm1d(hidden_size1)
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)
        self.bn2 = nn.BatchNorm1d(hidden_size2)
        self.fc3 = nn.Linear(hidden_size2, hidden_size3)
        self.bn3 = nn.BatchNorm1d(hidden_size3)
        self.fc4 = nn.Linear(hidden_size3, 1)
        self.dropout = nn.Dropout(0.3)

        # Tăng chuyển đổi để khớp kích thước cho skip connection
        self.skip_fc = nn.Linear(hidden_size1, hidden_size3) if hidden_size1 != hidden_size3 else None

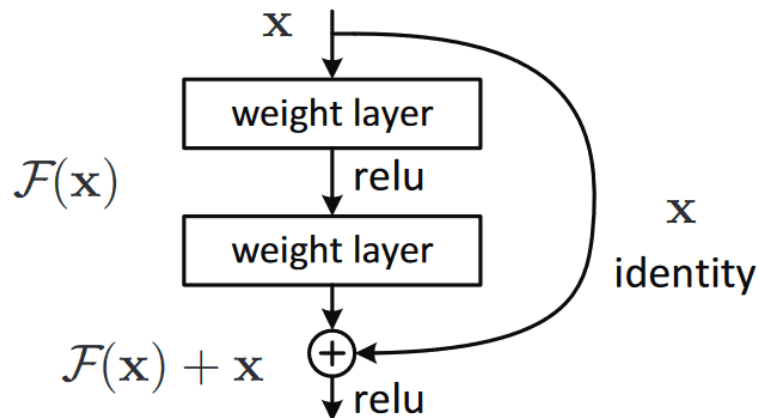
    def forward(self, x):
        # Tầng 1
        x1 = F.relu(self.bn1(self.fc1(x)))
        x1 = self.dropout(x1)

        # Tầng 2
        x2 = F.relu(self.bn2(self.fc2(x1)))
        x2 = self.dropout(x2)

        # Tầng 3 với skip connection từ tầng 1
        x3 = F.relu(self.bn3(self.fc3(x2)))
        if self.skip_fc is not None:
            x1_skip = self.skip_fc(x1) # Chuyển đổi kích thước để khớp
        else:
            x1_skip = x1
        x3 = x3 + x1_skip # Skip connection
        x3 = F.relu(x3)
        x3 = self.dropout(x3)

        # Tầng đầu ra
        out = self.fc4(x3)
        return out
```

Skip connections: là kỹ thuật cho phép đầu ra của một tầng được cộng trực tiếp vào đầu ra của một tầng sau. Điều này giúp giảm hiện tượng vanishing gradient và cho phép mô hình học được các đặc trưng ở nhiều cấp độ (cả nông và sâu).



Link: [Intuitive Explanation of Skip Connections in Deep Learning | AI Summer](#)

Đánh giá

Vì số lượng tầng ẩn nhóm em sử dụng giao động từ 2 đến 4 nên khá nông, vanishing gradient ít xảy ra nên Skip connections không mang lại cải thiện đáng kể so với MLP cơ bản. Vì vậy để đơn giản hóa và tránh tốn nhiều tài nguyên, nhóm em không sử dụng phương pháp này

2. Dùng đặc trưng khác cho mô hình

Thay vì thêm padding 0 để số lượng frame các mẫu bằng nhau. Nhóm em tính giá trị trung bình (mean) và độ lệch chuẩn (std) của 128 chiều của đặc trưng âm thanh (axis=0). Lúc này sẽ có 2 mảng có kích thước đều là (128,). Ghép nối chúng (mean, std) lại thành 1 mảng duy nhất có kích thước (256,0). Điều này giúp đảm bảo đặc trưng của mỗi mẫu có cùng kích thước là (256,)

```
# Lớp Dataset tùy chỉnh
class AudioDataset(Dataset):
    def __init__(self, data):
        self.data = data
        for sample in data:
            embedding = sample['audio_embedding']
            # Chuyển embedding thành tensor và chuẩn hóa về [0,1]
            embedding_tensor = torch.tensor(embedding, dtype=torch.float32) / 255.0
            # Tính mean và std theo trục thời gian
            mean = torch.mean(embedding_tensor, dim=0) # (128,)
            if len(embedding) == 1:
                std = torch.zeros(128, dtype=torch.float32)
            else:
                std = torch.std(embedding_tensor, dim=0) # (128,)
            # Ghép nối để tạo vector đặc trưng 256 chiều
            feature = torch.cat([mean, std], dim=0) # (256,)
            sample['feature'] = feature

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        return (sample['feature'],
                torch.tensor(sample['is_turkey'], dtype=torch.float32),
                sample['vid_id'])
```

D. Kết quả trên tập test

Cuối cùng, nhóm sẽ thực hiện dự đoán trên tập test. Kết quả sau khi submit của nhóm:



Mặc dù kết quả trên tập val của mô hình ở phase 2 này (accuracy 0.9665) lớn hơn kết quả của mô hình trên tập val của mô hình ở phase 1 (accuracy 0.9425) nhưng kết quả nộp trên kaggle thì mô hình ở phase 2 lại thấp hơn mô hình ở phase 1.

Lý do có thể là vì:

- Do cách chia val và train khác nhau nên có thể dữ liệu trên tập val ở phase 2 dễ học các đặc trưng hơn.
- Vì liên tục đánh giá trên tập val trong quá trình huấn luyện nên có thể mô hình đã “nhìn thấy” dữ liệu trên tập val, dẫn đến không tổng quát tốt

Đề xuất cách giải quyết:

- Sử dụng cross-validation
- Sử dụng nhiều tầng ẩn hơn, kết hợp thêm skip connection

Tham Khảo

[1] PyTorch Documentation. (2025). torch.optim — PyTorch 2.0 documentation. Available at: <https://pytorch.org/docs/stable/optim.html>

[2] Nguyễn, Q. (n.d.). Hiểu về Skip Connection: Một kỹ thuật nhỏ mà có võ trong các kiến trúc Residual Networks. Viblo. Available at: <https://viblo.asia/p/paper-explain-hieu-ve-skip-connection-mot-ki-thuat-nho-ma-co-vo-trong-cac-kien-truc-residual-networks-3Q75w7bQ5Wb>

[3] PyTorch. (n.d.). PyTorch Tutorials. Available at: <https://pytorch.org/tutorials/>

[4] Tjep, V. H. Multi-layer Perceptron. Machine Learning Cơ Bản. Available at: <https://machinelearningcoban.com/2017/02/24/mlp/>